

Name

Class

Date

OOP - 8

Total questions: 10

Worksheet time: 6mins

1.

```
class A {
    int data;
public:
    A() {cout << "A() ";}
    A(const A& other) {cout << "A(other&) ";}
    A& operator=(const A& other) {cout << "Op= A(other&) ";}
};

class B {
    A obj;
public:
    B() {cout << "B() ";}
    B(const B& other) {cout << "B(other&) ";}
    B& operator=(const B& other) {cout << "Op= B(other&) ";}
};

int main() {
    B b1;
    B b2 = b1;
}
```

a) A() B() A(other&) B(other&)

b) A() B() A() B(other&)

c) B() A() B(other&) A()

d) A() B() Op= B(other&)

e) Няма да се компилира!

2.

```
class A {
    int data;
public:
    A() {cout << "A() ";}
    A(int _data) : data(_data) {cout << "A(data) ";}
    A(const A& other) {cout << "A(other&) ";}
    A(A&& other) {cout << "A(other&&) ";}
    A& operator=(const A& other) {cout << "Op= A(other&) ";}
    A& operator=(A&& other) {cout << "Op= A(other&&) ";}
    ~A() {cout << "~A() ";}
};

int main() {
    const A a1(4);
    A a2(std::move(a1));
}
```

a) A(data) A(other&&) ~A()

b) A() A() Op= A(other&) ~A() ~A()

c) A(data) A(other&) ~A() ~A()

d) A(data) A(other&&) ~A() ~A()

e) Няма да се компилира!

3. Какви са предимствата на масив от указатели към обекти пред масив от обекти? (може да има повече от един верен отговор)

a) Позволява "празни позиции" в масива (чрез nullptr).

b) Гарантира винаги по-бърз достъп до елементите.

c) Премахва нуждата от динамично заделяне на памет.

d) Намалява копирането на обекти при оразмеряване на масива.

e) Заема по-малко памет във всички случаи.

4. Какви са предимствата на динамичните масиви пред статичните масиви? (може да има повече от един верен отговор)

- a) Размерът им може да се определя по време на изпълнение на програмата.
- b) Винаги заемат по-малко памет от статичните масиви.
- c) Достъпът до елементите им е винаги по-бърз.
- d) Не е необходимо да се заделя памет за тях.
- e) Могат да се оразмеряват по време на изпълнение на програмата.

5. Какво е smart pointer?

- a) Клас, който инкапсулира суров указател, управлява неговия живот и дефинира ясна ownership семантика.
- b) Функция за заделяне на динамична памет.
- c) Обикновен указател, който може да сочи само към един обект.
- d) Масив от указатели към обекти.

6. Кой smart pointer позволява само един собственик на даден обект?

- a) `std::shared_ptr`
- b) `std::weak_ptr`
- c) `std::unique_ptr`
- d) `std::auto_ptr`
- e) Няма да се компилира! 🙄

7. Кое твърдение за `std::unique_ptr` е вярно?

- a) Може свободно да се копира.
- b) Може да има много собственици едновременно.
- c) Не може да се копира, но може да се премества.
- d) Изисква ръчно delete.

8. Кой smart pointer НЕ увеличава брояча на референции на `shared_ptr`?

- a) `std::unique_ptr`
- b) `std::shared_ptr`
- c) `std::auto_ptr`
- d) `std::weak_ptr`

9. Кога обект, управляван от `std::shared_ptr`, се освобождава от паметта?

- a) Когато всички копия на `shared_ptr` бъдат унищожени.
- b) Веднага след създаването му.
- c) Само при ръчно извикване на `delete`.
- d) Когато първият `pointer` излезе от `scope`.
- e) Подвеждащ въпрос. Той никога не се освобождава.

10. Кое твърдение за `std::weak_ptr` е вярно?

- a) Увеличава `reference count`-а на обекта.
- b) Може директно да се `dereference`-ва.
- c) Не притежава обекта и не влияе на `lifetime`-а му.
- d) Винаги предотвратява изтриването на обекта.

Answer Keys

1. b) A() B() A() B(other&) 2. c) A(data) A(other&) ~A()
~A()
3. a) Позволява , Намалява
"празни позиции" в масива (чрез nullptr).
d) копирането на обекти при оразмеряване на масива.
4. a) Размерът , Могат да се
им може да се е) оразмеряват
се по време на изпълнение
определя изпълнение по време на изпълнение
на програмата.
5. a) Клас, който
инкапсулира суров указател, управлява
неговия живот и дефинира ясна
ownership семантика.
6. c) std::unique_ptr
7. c) Не може да се копира,
но може да се премества.
8. d) std::weak_ptr
9. a) Когато всички копия
на shared_ptr бъдат унищожени.
10. c) Не притежава обекта
и не влияе на lifetime-
а му.

